



CRUD amallarini yaratish. CRUD asosida mini loyiha

Maktab blogini to'liq CRUD ilovaga aylantirish: ro'yxat (sahifalash bilan), batafsil ko'rish, yaratish, tahrirlash, o'chirish (tasdiqlash bilan), faqat muallifga ruxsat; xuddi shu ishni Class Based View'lar (ListView, DetailView, CreateView, UpdateView, DeleteView) bilan qilish; sinovdan o'tkazish.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Model, View, Template va Form'ni bitta to'liq ishlaydigan ilovaga **birlashtirishni**;
- To'rtta CRUD amalini veb-interfeys orqali amalga oshirishni;
- **Ruxsatlarni**: maqolani faqat uning muallifi tahrirlay va o'chira olishini;
- Uzun ro'yxatni sahifalarga bo'lishni (pagination);
- Xuddi shu ilovani Django'ning tayyor **Class Based View**'lari bilan bir necha qatorda yozishni;
- Loyihani sinovdan o'tkazishni (21-loyiha: Blog veb-ilovasi).

Reja: CRUD = 5 ta sahifa

Amal	URL	View	Shablon
R — ro'yxat	/	maqola_royxat	maqola_list.html
R — batafsil	/maqola/<id>/	maqola_batafsil	maqola_detail.html
C — yaratish	/maqola/yangi/	maqola_yaratish	maqola_form.html

U — tahrirlash	/maqola/<id>/tahrirlash/	maqola_tahrirlash	maqola_form.html (o'sha!)
D — o'chirish	/maqola/<id>/ochirish/	maqola_ochirish	maqola_confirm_delete.html

Bu jadval — har qanday CRUD ilovaning "retsepti". Kutubxona, talabalar tizimi, mahsulotlar — hammasi xuddi shu 5 ta sahifadan iborat.

1. URL'lar

blog/urls.py

```
from django.urls import path
from . import views

app_name = "blog"

urlpatterns = [
    path("", views.maqola_royxat, name="bosh"),
    path("maqola/yangi/", views.maqola_yaratish, name="yaratish"),          #
    <int:id> dan OLDIN!
    path("maqola/<int:id>/", views.maqola_batafsil, name="maqola"),
    path("maqola/<int:id>/tahrirlash/", views.maqola_tahrirlash,
name="tahrirlash"),
    path("maqola/<int:id>/ochirish/", views.maqola_ochirish, name="ochirish"),
    path("kategoriya/<slug:nomi>/", views.kategoriya, name="kategoriya"),
    path("qidiruv/", views.qidiruv, name="qidiruv"),
    path("royxat/", views.royxatdan_otish, name="royxat"),
]
```

2. Forma

blog/forms.py

```
from django import forms
from .models import Maqola

class MaqolaForm(forms.ModelForm):
    class Meta:
        model = Maqola
        fields = ["sarlavha", "kategoriya", "matn", "chop_etilgan"]
        widgets = {"matn": forms.Textarea(attrs={"rows": 10})}

    def clean_sarlavha(self):
        s = self.cleaned_data["sarlavha"].strip()
        if len(s) < 5:
            raise forms.ValidationError("Sarlavha kamida 5 belgi bo'lsin")
        return s
```

3. View'lar (funksiya asosida)

blog/views.py

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.core.paginator import Paginator
from django.db.models import F
from django.http import HttpResponseForbidden
from django.shortcuts import get_object_or_404, redirect, render

from .forms import MaqolaForm
from .models import Maqola

# ----- READ: ro'yxat -----
def maqola_royxat(request):
    maqolalar = (Maqola.objects.filter(chop_etilgan=True)
                  .select_related("kategoriya", "muallif"))
    sahifalovchi = Paginator(maqolalar, 6) # har sahifada 6 ta
    sahifa = sahifalovchi.get_page(request.GET.get("page")) # ?page=2
    return render(request, "blog/maqola_list.html", {"sahifa": sahifa})

# ----- READ: batafsil -----
def maqola_batafsil(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    # Chop etilmagan maqolani faqat muallifi ko'ra oladi
    if not maqola.chop_etilgan and maqola.muallif != request.user:
        return HttpResponseForbidden("Bu maqola hali chop etilmagan")
    Maqola.objects.filter(id=id).update(korishlar=F("korishlar") + 1)
    return render(request, "blog/maqola_detail.html", {"maqola": maqola})

# ----- CREATE -----
@login_required
def maqola_yaratish(request):
    if request.method == "POST":
        form = MaqolaForm(request.POST)
        if form.is_valid():
            maqola = form.save(commit=False)
            maqola.muallif = request.user
            maqola.save()
```

```

        messages.success(request, "✅ Maqola yaratildi")
        return redirect(maqola) #
get_absolute_url() ga
    else:
        form = MaqolaForm()
        return render(request, "blog/maqola_form.html", {"form": form, "sarlavha":
"Yangi maqola"})

# ----- UPDATE -----
@login_required
def maqola_tahrirlash(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if maqola.muallif != request.user: # 🔒 faqat muallif
        return HttpResponseForbidden("Siz bu maqolani tahrirlay olmaysiz")
    if request.method == "POST":
        form = MaqolaForm(request.POST, instance=maqola) # instance – mavjud
obyektni yangilash
        if form.is_valid():
            form.save()
            messages.success(request, "✅ O'zgarishlar saqlandi")
            return redirect(maqola)
        else:
            form = MaqolaForm(instance=maqola) # forma eski
            qiyamatlar bilan to'lgan
            return render(request, "blog/maqola_form.html", {"form": form, "sarlavha":
"Tahrirlash"})

# ----- DELETE -----
@login_required
def maqola_ochirish(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if maqola.muallif != request.user:
        return HttpResponseForbidden("Siz bu maqolani o'chira olmaysiz")
    if request.method == "POST": # faqat
tasdiqlangandan keyin
        maqola.delete()
        messages.success(request, "🗑️ Maqola o'chirildi")
        return redirect("blog:bosh")
    return render(request, "blog/maqola_confirm_delete.html", {"maqola":
maqola})

```



Xavfsizlik: ikki qoida

1. **Ruxsatni serverda tekshiring.** Shablonda "Tahrirlash" tugmasini yashirish yetarli emas — ayyor foydalanuvchi `/maqola/5/tahrirlash/` manzilini qo'lda yozishi mumkin. Shuning uchun view'da `if maqola.muallif != request.user`.
2. **O'chirish — faqat POST.** Agar GET so'rov bilan o'chirsak, oddiy havola (yoki boshqa saytdagi rasm teg!) maqolani o'chirib yuborishi mumkin. Avval tasdiqlash sahifasi, keyin POST forma + CSRF.

`MaqolaForm(request.POST, instance=maqola)` — **yaratish va tahrirlash** uchun bitta forma va bitta shablon: `instance` berilsa — mavjud obyekt yangilanadi, berilmasa — yangisi yaratiladi.

4. Shablonlar

blog/templates/blog/maqola_list.html

```
{% extends "blog/base.html" %}
{% block title %}Maktab blogi{% endblock %}

{% block content %}
<div class="sarlavha-qator">
  <h1>So'nggi maqolalar</h1>
  {% if user.is_authenticated %}
    <a class="tugma" href="{% url 'blog:yaratish' %}">👉 Yangi maqola</a>
  {% endif %}
</div>

<div class="kartalar">
  {% for m in sahifa %}
    {% include "blog/_karta.html" %}
  {% empty %}
    <p>Hozircha maqola yo'q. Birinchisini siz yozing!</p>
  {% endfor %}
</div>

{% if sahifa.has_other_pages %}
<nav class="sahifalash">
  {% if sahifa.has_previous %}
    <a href="?page={{ sahifa.previous_page_number }}">← Oldingi</a>
  {% endif %}
  <span>{{ sahifa.number }} / {{ sahifa.paginator.num_pages }}</span>
  {% if sahifa.has_next %}
    <a href="?page={{ sahifa.next_page_number }}">Keyingi →</a>
  {% endif %}
</nav>
{% endif %}
{% endblock %}
```

```

{% extends "blog/base.html" %}
{% block title %}{{ maqola.sarlavha }}{% endblock %}

{% block content %}
    {% for msg in messages %}<p class="xabar">{{ msg }}</p>{% endfor %}

    <article class="maqola">
        <a class="belgi" href="{% url 'blog:kategoriya' maqola.kategoriya.slug %}">
            {{ maqola.kategoriya }}</a>
        <h1>{{ maqola.sarlavha }}</h1>
        <p class="meta">👤 {{ maqola.muallif.username }} · 📅 {{
maqola.yaratilgan|date:"d.m.Y H:i" }} · 👁 {{ maqola.korishlar }}</p>
        {% if not maqola.chop_etilgan %}<p class="ogohlantirish">✎ Qoralama –
faqat siz ko'ryapsiz</p>{% endif %}
        {{ maqola.matn|linebreaks }}

        {% if user == maqola.muallif %}                                {# tugmalar faqat
muallifga ko'rinadi #}
            <div class="amallar">
                <a class="tugma" href="{% url 'blog:tahrirlash' maqola.id %}">✎
Tahrirlash</a>
                <a class="tugma qizil" href="{% url 'blog:ochirish' maqola.id %}">🗑
O'chirish</a>
            </div>
        {% endif %}
    </article>
    <a href="{% url 'blog:bosh' %}">← Barcha maqolalar</a>
{% endblock %}

```

```
{% extends "blog/base.html" %}
{% block title %}{{ sarlavha }}{% endblock %}

{% block content %}
<h1>{{ sarlavha }}</h1>
<form method="post" class="forma">
    {% csrf_token %}
    {% for field in form %}
        <div class="maydon">
            {{ field.label_tag }}
            {{ field }}
            {% for xato in field.errors %}<small class="xato">{{ xato }}</small>{%
endfor %}
        </div>
    {% endfor %}
    <button type="submit" class="tugma">💾 Saqlash</button>
    <a href="{% url 'blog:bosh' %}">Bekor qilish</a>
</form>
{% endblock %}
```

```
{% extends "blog/base.html" %}
{% block content %}
<h1>O'chirishni tasdiqlang</h1>
<p><<<b>{{ maqola.sarlavha }}</b>>> maqolasini o'chirmoqchimisiz? Bu amalni
qaytarib bo'lmaydi.</p>
<form method="post">
    {% csrf_token %}
    <button type="submit" class="tugma qizil">Ha, o'chirish</button>
    <a href="{{ maqola.get_absolute_url }}">Yo'q, ortga</a>
</form>
{% endblock %}
```

style.css ga qo'shimcha

```
.sarlavha-qator { display: flex; justify-content: space-between; align-items: center; }
.tugma { display: inline-block; background: #4f46e5; color: #fff; padding: 10px 18px; border-radius: 10px;
        text-decoration: none; border: none; font: inherit; cursor: pointer; }
.tugma.qizil { background: #dc2626; }
.forma .maydon { display: grid; gap: 6px; margin-bottom: 16px; }
.forma input, .forma select, .forma textarea { padding: 10px; border: 2px solid #e5e7eb; border-radius: 10px; font: inherit; }
.xato { color: #dc2626; }
.xabar { background: #dcfce7; padding: 12px; border-radius: 10px; }
.sahifalash { display: flex; gap: 16px; justify-content: center; margin: 30px 0; }
.amallar { display: flex; gap: 10px; margin: 20px 0; }
```

5. Xuddi shu narsa — Class Based View'lar bilan

CRUD shunchalik keng tarqalganki, Django uning uchun **tayyor klasslar** beradi. Yuqoridagi ~80 qator view kodi ~40 qatorga qisqaradi:

```

from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.contrib.messages.views import SuccessMessageMixin
from django.urls import reverse_lazy
from django.views.generic import ListView, DetailView, CreateView, UpdateView,
DeleteView

from .forms import MaqolaForm
from .models import Maqola

class MaqolaList(ListView):
    queryset =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    paginate_by = 6 # shablonda: page_obj, object_list
    # shablon (standart): blog/maqola_list.html

class MaqolaDetail(DetailView):
    model = Maqola
    pk_url_kwarg = "id" # URL'da <int:id> ishlatganimiz
    uchun
    # shablon: blog/maqola_detail.html, obyekt: {{ object }} yoki {{ maqola }}

class MaqolaCreate(LoginRequiredMixin, SuccessMessageMixin, CreateView):
    model = Maqola
    form_class = MaqolaForm
    success_message = "✅ Maqola yaratildi"
    extra_context = {"sarlavha": "Yangi maqola"} # shablondagi {{ sarlavha
    }} uchun

    def form_valid(self, form):
        form.instance.muallif = self.request.user
        return super().form_valid(form) # saqlash va get_absolute_url() ga
        o'tish

class FaqatMuallif(UserPassesTestMixin):
    """Faqat muallifga ruxsat beruvchi 'aralashma' (mixin)."""
    def test_func(self):
        return self.get_object().muallif == self.request.user

```

```

class MaqolaUpdate(LoginRequiredMixin, FaqatMuallif, SuccessMessageMixin,
UpdateView):
    model = Maqola
    form_class = MaqolaForm
    pk_url_kwarg = "id"
    success_message = "✅ O'zgarishlar saqlandi"
    extra_context = {"sarlavha": "Tahrirlash"}

class MaqolaDelete(LoginRequiredMixin, FaqatMuallif, DeleteView):
    model = Maqola
    pk_url_kwarg = "id"
    success_url = reverse_lazy("blog:bosh")

```

blog/urls.py (CBV varianti)

```

from . import views_cbv as v

urlpatterns = [
    path("", v.MaqolaList.as_view(), name="bosh"),
    path("maqola/yangi/", v.MaqolaCreate.as_view(), name="yaratish"),
    path("maqola/<int:id>/", v.MaqolaDetail.as_view(), name="maqola"),
    path("maqola/<int:id>/tahrirlash/", v.MaqolaUpdate.as_view(),
name="tahrirlash"),
    path("maqola/<int:id>/ochirish/", v.MaqolaDelete.as_view(),
name="ochirish"),
]

```



CBV shablon nomlari

Tayyor klasslar shablonni **avtomatik** nomi bo'yicha qidiradi: `ilova/model_list.html`, `model_detail.html`, `model_form.html`, `model_confirm_delete.html`. Shuning uchun biz shablonlarni oldindan aynan shunday nomladik! `ListView` da ro'yxat — `page_obj` (yoki `maqola_list`), shuning uchun `maqola_list.html` dagi sahifa ni `page_obj` ga almashtiring.

Klass

Nima qiladi

`ListView`

Ro'yxat + sahifalash

<code>DetailView</code>	Bitta obyekt (topilmasa 404)
<code>CreateView</code>	GET — bo'sh forma, POST — tekshirish va yaratish
<code>UpdateView</code>	Eski qiymatlar bilan forma, saqlash
<code>DeleteView</code>	Tasdiqlash sahifasi va POST bilan o'chirish
<code>LoginRequiredMixin</code>	<code>@login_required</code> ning CBV varianti
<code>UserPassesTestMixin</code>	<code>test_func()</code> True qaytarmasa — 403

6. Sinovdan o'tkazish

Loyihani topshirishdan oldin har bir ssenariyni tekshiring. Ikki xil brauzerda (yoki biri "Inkognito" rejimida) ikki foydalanuvchi bilan kiring:

#	Ssenariy	Kutilgan natija	✓
1	Mehmon bosh sahifani ochadi	Chop etilgan maqolalar, "Yangi maqola" tugmasi yo'q	☐
2	Mehmon <code>/maqola/yangi/</code> ni qo'lda ochadi	Login sahifasiga yo'naltiriladi	☐
3	Ali maqola yaratadi (sarlavha 3 belgi)	"Kamida 5 belgi" xatosi, forma ma'lumotlari yo'qolmaydi	☐
4	Ali to'g'ri maqola yaratadi	Maqola sahifasi + yashil xabar; admin'da muallif — Ali	☐
5	Vali Ali'ning maqolasini ochadi	Tahrirlash/o'chirish tugmalari ko'rinmaydi	☐
6	Vali <code>/maqola/ID/tahrirlash/</code> ni qo'lda ochadi	403 Forbidden	☐
7	Ali maqolani o'chiradi	Tasdiqlash sahifasi → "Ha" → ro'yxatda yo'q	☐

Avtomatik test (qisqacha tanishuv)

blog/tests.py

```
from django.contrib.auth.models import User
from django.test import TestCase
from django.urls import reverse
from .models import Kategoriya, Maqola

class MaqolaTestlari(TestCase):
    def setUp(self):
        self.ali = User.objects.create_user("ali", password="parol12345")
        self.vali = User.objects.create_user("vali", password="parol12345")
        kat = Kategoriya.objects.create(nomi="Sport", slug="sport")
        self.m = Maqola.objects.create(sarlavha="Test maqola", matn="...",
            kategoriya=kat, muallif=self.ali)

    def test_bosh_sahifa_ochiladi(self):
        javob = self.client.get(reverse("blog:bosh"))
        self.assertEqual(javob.status_code, 200)
        self.assertContains(javob, "Test maqola")

    def test_begona_tahrirlay_olmaydi(self):
        self.client.login(username="vali", password="parol12345")
        javob = self.client.get(reverse("blog:tahrirlash", args=[self.m.id]))
        self.assertEqual(javob.status_code, 403)
```

Terminal

```
python manage.py test blog
```

Natija

```
Creating test database for alias 'default'...
```

```
..
```

```
-----  
Ran 2 tests in 0.412s
```

```
OK
```

Testlar alohida, vaqtinchalik bazada ishlaydi — haqiqiy ma'lumotlaringizga tegmaydi.

Amaliy mashg'ulot (21-loyiha: Blog veb-ilovasi)

- 1 Yuqoridagi URL, forma, view va shablonlarni loyihangizga qo'shing (funksiya variantida).
- 2 Sinov jadvalidagi 9 ta ssenariyni bajaring va belgilang.
- 3 View'larni CBV variantiga o'tkazing va ssenariylarni qayta tekshiring.
- 4 "Mening maqolalarim" sahifasi: `/mening/` — joriy foydalanuvchining barcha maqolalari (qoralamalar bilan).
- 5 Loyihani GitHub'ga push qiling; README'da: loyiha tavsifi, texnologiyalar, ishga tushirish yo'riqnomasi, skrinshotlar.

▼ 4-topshiriq yechimi

Python

```
@login_required  
def mening_maqolalarim(request):  
    maqolalar =  
    Maqola.objects.filter(muallif=request.user).select_related("kategoriya")  
    return render(request, "blog/mening.html", {"maqolalar": maqolalar})  
  
# urls.py: path("mening/", views.mening_maqolalarim, name="mening"),
```

Ko'p uchraydigan xatolar

- ✗ `maqola/yangi/` yo'li `maqola/<int:id>/` dan **keyin** yozilsa — muammo yo'q (chunki "yangi" son emas), lekin `<str:...>` ishlatilsa — "yangi" so'zi id deb olinadi. Aniq yo'llarni doim oldin yozing.
- ✗ Tahrirlashda `instance=` unutilgan — yangi nusxa yaratiladi, eskisi o'zgarmaydi.
- ✗ Ruxsat faqat shablonda tekshirilgan — URL'ni qo'lda yozib, boshqa bironing maqolasini o'chirish mumkin bo'lib qoladi.
- ✗ GET so'rov bilan o'chirish.
- ✗ `TemplateDoesNotExist: blog/maqola_form.html` — CBV standart nomli shablonni qidirayпти; faylni shunday nomlang yoki `template_name` bering.

O'zingizni tekshiring

? ModelForm bilan mavjud obyektни tahrirlash uchun formaga nima beriladi?

- A. `id=`
- B. `object=`
- C. `instance=`
- D. `update=True`

? Nima uchun o'chirish faqat POST so'rov bilan bo'lishi kerak?

- A. POST tezroq ishlaydi
- B. Oddiy havola yoki begona sayt tasodifan/ataylab o'chirib yubormasligi uchun (CSRF himoyasi bilan)
- C. GET so'rovda id bo'lmaydi
- D. Django GET'ni qo'llab-quvvatlamaydi

? "Faqat muallif tahrirlay oladi" tekshiruvi qayerda bo'lishi SHART?

- A. View'da (serverda)
- B. Faqat shablonda
- C. CSS'da
- D. JavaScript'da

? Ro'yxatni sahifalarga bo'luvchi Django klassi?

- A. `Splitter`
- B. `PageView`
- C. `Limiter`
- D. `Paginator`

? Yangi obyekt yaratuvchi tayyor CBV?

- A. `ListView`
- B. `CreateView`
- C. `NewView`
- D. `FormView`

? CBV'da tizimga kirishni talab qiluvchi aralashma (mixin)?

- A. `AuthMixin`
- B. `UserPassesTestMixin`
- C. `LoginRequiredMixin`
- D. `SessionMixin`



Uyga vazifa (IV bob yakunlovchi loyiha)

Quyidagilardan birini to'liq CRUD ilova sifatida yarating (model + admin + forma + 5 sahifa + ruxsatlar + sahifalash + kamida 2 ta test):

1. **Kutubxona boshqaruv tizimi** (22-loyiha): kitoblar CRUD'i, kitobni "olish/qaytarish".
2. **Talabalar boshqaruv tizimi** (23-loyiha): talabalar, guruhlar, baholar; guruh bo'yicha o'rtacha baho.
3. **E'lonlar taxtasi**: foydalanuvchilar e'lon joylaydi (rasm bilan), toifa va qidiruv bo'yicha filtr.



IV bob yakunlandi!

- CRUD = ro'yxat, batafsil, yaratish, tahrirlash, o'chirish — 5 ta URL/view/shablon.
- `ModelForm(request.POST, instance=obj)` — yaratish va tahrirlash uchun bitta forma.
- Ruxsatlar serverda; o'chirish — tasdiqlash + POST.
- `Paginator`; tayyor CBV'lar: `ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView` + mixinlar.
- Siz endi to'liq ishlaydigan, xavfsiz veb-ilova yarata olasiz! Keyingi bobda unga **API** qo'shamiz va **internetga chiqaramiz**.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. ModelForm bilan mavjud obyektни tahrirlash uchun formaga nima beriladi? — **C) instance=**
2. Nima uchun o'chirish faqat POST so'rov bilan bo'lishi kerak? — **B) Oddiy havola yoki begona sayt tasodifan/ataylab o'chirib yubormasligi uchun (CSRF himoyasi bilan)**
3. "Faqat muallif tahrirlay oladi" tekshiruvi qayerda bo'lishi SHART? — **A) View'da (serverda)**
4. Ro'yxatni sahifalarga bo'luvchi Django klassi? — **D) Paginator**
5. Yangi obyekt yaratuvchi tayyor CBV? — **B) CreateView**
6. CBV'da tizimga kirishni talab qiluvchi aralashma (mixin)? — **C) LoginRequiredMixin**